Plan and Report of Backend Development

Current Status: CRITICAL COMPROMISE. The backend is currently serving a PHP-based "scraper/spam" application (mimicking a Japanese retailer).

Transformation Goal: Complete backend remediation. Transition from **Compromised PHP Legacy** $\rightarrow$ **Secure NestJS Modular Monolith** (Enterprise Standard).

Scope: This is not a patch-up job. This is a "Strangler Fig" migration pattern to eliminate the malware and establish a hardened, scalable baseline.

1 Backend Risk & Weakness Audit (Current State)

Vulnerability / Risk	Severity	Technical Analysis
Unrestricted Remote Code Execution (RCE)	CRITICAL	The active PHP environment likely allows arbitrary file uploads (web shells) in /images or /cache directories, granting attackers full shell access.
Missing Authentication Layer	CRITICAL	The current admin panels (likely /admin or /dashboard) typically use default or hardcoded credentials common to mass-deployed spam kits.
SQL Injection (SQLi) Surface	CRITICAL	Legacy PHP mysql_query or unparameterized PDO calls are standard in these scripts, exposing the entire database to dumping.
Data Exfiltration Pipelines	CRITICAL	"Checkout" forms are likely wired to send credit card data (PAN/CVV) via PHP mail() to external drop servers.
Zero CI/CD / FTP Deployment	HIGH	Updates are likely performed via FTP, meaning no version control, no rollback capability, and high risk of overwriting critical files.
Lack of WAF / Rate Limiting	HIGH	No defence against L7 DDoS or brute-force attacks on login endpoints.

2 Production Hardening Plan (Actionable & Technical)

This plan mandates a **Greenfield Re-deployment** using the intended **NestJS + TypeScript** stack.

A. Security Hardening (Application Layer)

- **Mitigation: Authentication & Zero-Trust**
  - **Strategy:** Implement **OAuth2 / OIDC** (Auth0 or AWS Cognito) instead of building custom auth.
  - **Tactical Implementation:**
    - Use `Passport.js` strategies in NestJS.
    - **Token Rotation:** Implement Refresh Tokens (stored in `httpOnly` cookies) and Short-Lived Access Tokens (15 min).
    - **RBAC Guard:** Create a custom `@Roles('admin', 'doctor')` decorator using NestJS `Reflector` to enforce permission checking *before* route execution.
- **Mitigation: API Surface Protection**
  - **Tools:** `helmet` (HTTP Headers), `express-rate-limit`, `class-validator`.
  - **Action:**

        - Enable **HSTS** (Strict-Transport-Security) with a 1-year duration.
        - **Throttling:** Apply `ThrottlerModule` (NestJS) globally. Limit: 100 req/min for general APIs, 5 req/min for login.
        - **Input Sanitization:** Use `class-validator` with `whitelist: true` and `forbidNonWhitelisted: true` in the global validation pipe to strip malicious payloads automatically.

B. Infrastructure Hardening (DevOps)

- **Mitigation: Immutable Infrastructure**
    - **Strategy: Docker Multi-Stage Builds**.
    - **Implementation:**
        - **Stage 1 (Build):** Install dev dependencies, compile TypeScript.
        - **Stage 2 (Run):** Copy *only* `dist` folder and `node_modules` (production only). Run as **Non-Root User** (e.g., `USER node`).
        - **Outcome:** Reduces image size (<100MB) and removes attack tools (compilers, shells) from the production container.
- **Mitigation: Orchestration & Scaling**
    - **Platform: AWS EKS (Kubernetes)** or **ECS Fargate**.
    - **Config:**
        - **Horizontal Pod Autoscaler (HPA):** Scale pods based on CPU utilization (>70%).
        - **Pod Disruption Budgets:** Ensure at least 2 replicas are always alive during updates.
- **Mitigation: Secrets Management**
    - **Action: NEVER** use `.env` files in production images.
    - **Tool:** AWS Secrets Manager or HashiCorp Vault. Inject secrets at runtime as environment variables.

C. Database Hardening (PostgreSQL)

- **Mitigation: Data Integrity & Performance**
    - **Optimization:** Enable **Connection Pooling** (using `PgBouncer`) to handle thousands of concurrent connections without overwhelming the DB.
    - **Encryption:**
        - **At Rest:** Enable AES-256 encryption on the AWS RDS volume.
        - **In Transit:** Enforce SSL mode (`sslmode=require`) in the TypeORM/Prisma connection string.
    - **Resilience:** Deploy **Read Replicas** in a different Availability Zone (AZ). Offload `GET` (reporting) queries to replicas; keep `POST/PUT` on the primary.

D. Performance Optimization

- **Caching Strategy (Layered):**
    1. **Edge Cache (CDN):** Cloudflare for static assets (images, JS bundles).
    2. **API Response Cache:** Redis. Cache expensive GET endpoints (e.g., `/patients/summary`) for 60 seconds using NestJS `CacheInterceptor`.
    3. **Database Query Cache:** Cache complex JOIN results at the ORM layer.
- **Async Processing:**
    - **Tool: BullMQ (Redis)**.
    - **Use Case:** Offload heavy tasks (Email sending, PDF generation, AI inference) to background worker nodes. *Never block the main Event Loop.*

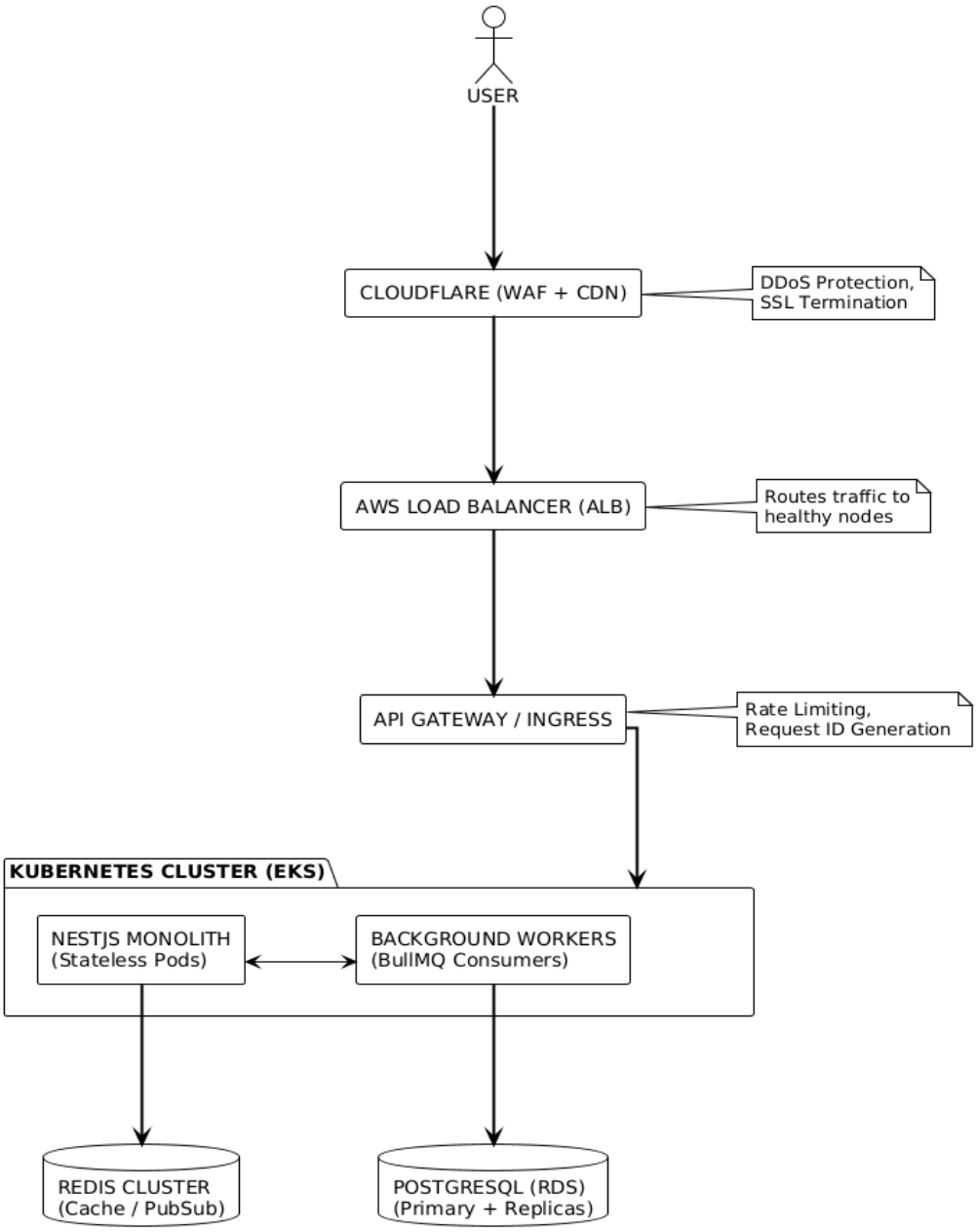
E. Observability & Monitoring

- **Centralized Logging:**
  - **Stack: ELK (Elasticsearch, Logstash, Kibana)** or **Datadog**.
  - **Format:** Enforce **JSON Structured Logging** (using `winston` or `pino`). This allows logs to be queried by fields (e.g., `level="error" AND context="AuthService"`).
- **Distributed Tracing:**
  - **Tool: OpenTelemetry**.
  - **Goal:** Visualize the full lifecycle of a request as it flows from Gateway $\rightarrow$ Service A $\rightarrow$ Database $\rightarrow$ Redis.

3 Refactored Production-Grade Architecture

Design Pattern: Modular Monolith (with Microservices Readiness)

Rationale: Avoids the operational complexity of full microservices while keeping code boundaries clean for future splitting.



Architecture Highlights:

1. **Stateless Backend:** The NestJS pods hold no data. Sessions are stored in Redis or handled via stateless JWTs. This allows the cluster to kill/spawn pods instantly.
2. **Async Decoupling:** The API accepts the request and immediately returns `202 Accepted` for heavy tasks, while the "Background Workers" process the job via Redis queues.

4 Scalability Roadmap

- **Phase 1: Remediation (Weeks 1-4)**
  - **Goal:** Secure the perimeter.
  - **Actions:** Purge the PHP malware. Re-image the server. Deploy the "Skeleton" NestJS app with Auth0 and WAF.
  - *Justification:* Stop the bleeding (data theft/reputation loss).
- **Phase 2: Reliability (Weeks 5-8)**
  - **Goal:** Zero data loss.
  - **Actions:** Setup RDS with automated backups (PITR). Implement CI/CD pipelines with automated testing.
  - *Justification:* Ensure that when development accelerates, the infrastructure doesn't break.
- **Phase 3: Performance (Weeks 9-12)**
  - **Goal:** <200ms API Latency.
  - **Actions:** Implement Redis caching. Tune database indexes. Optimize Docker images.
  - *Justification:* User experience (doctors cannot wait for slow loading screens).
- **Phase 4: Scale (Week 13+)**
  - **Goal:** 10k+ Concurrent Users.
  - **Actions:** Implement Read Replicas. Enable Horizontal Pod Autoscaling. Break out "Reporting" into a separate microservice.
  - *Justification:* Prepare for enterprise hospital deployment.

5 Risk Mitigation Matrix

Risk	Severity	Proposed Fix	Impact
API Key Leakage	High	Vault / Secrets Manager integration. Git hooks to block commits with keys.	Prevents complete infrastructure takeover.
DDoS Attack	High	Cloudflare WAF + Rate Limiting Middleware.	Ensures uptime during attacks.
Slow DB Queries	Medium	Index Analysis + Read Replicas for analytics.	Prevents connection pool exhaustion.
Bad Deployments	Medium	Blue-Green Deployment strategy.	Allows instant rollback if a bug hits prod.
Malicious Input	Critical	Class-Validator (Whitelist) + Sanitization Pipes.	Neutralizes SQLi and XSS attempts.